Evaluación de Unidad

1. Datos Generales

Asignatura	Álgebra Lineal
Ciclo	1 A
Unidad	1
Resultado de aprendizaje de la unidad	Aplica y resuelve analíticamente utilizando diversos métodos sistemas de ecuaciones lineales, matrices y determinantes, aplicaciones lineales según diversos criterios y utilizando diversas técnicas para la resolución de problemas con ayuda de software matemático, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad
Nombre del Docente	Edwin René Guamán Quinche
Nombre del Estudiante	Brigith de los Angeles Toledo Jumbo
Fecha	04 al 10 de noviembre de 2026
Horario	7:30-8:30 9:30-10:30 11:30-12:30

2. Objetivo:

- Demostrar la capacidad de identificar, modelar matemáticamente y resolver un problema práctico utilizando el álgebra lineal aplicando sistemas de ecuaciones lineales o matrices.

3. Contexto

En el presente proyecto se desarrolló un juego de Tres en Raya mediante el uso de matrices, aplicando los conocimientos adquiridos en la materia de Álgebra Lineal. El tablero está representado por una matriz 3x3, en el que dos jugadores colocan sus símbolos (X y O) por turnos, con el objetivo de conseguir tres símbolos iguales en una misma fila, columna o diagonal. Se eligió este tema porque es un juego sencillo y conocido que permite comprender cómo se pueden utilizar las operaciones matriciales en una situación práctica.

Es importante para la carrera de Computación porque permite relacionar los conceptos matemáticos con la programación y el desarrollo de aplicaciones. Ya que las matrices son herramientas utilizadas en diferentes áreas de la computación como: el procesamiento de imágenes, los videojuegos, la inteligencia artificial y el almacenamiento de datos. Durante el desarrollo de esta actividad se busca aprender a representar un tablero con matrices, realizar operaciones de suma y resta, identificar posiciones mediante índices y comprobar las condiciones necesarias para determinar un jugador.

4. Desarrollo de las operaciones

4.1. Planteamiento del problema

Se desea desarrollar un juego de Tres en Raya para dos jugadores, utilizando matrices de 3x3 para representar el tablero y registrar las jugadas. El objetivo es determinar cuándo uno de los jugadores consigue 3 símbolos iguales en una misma fila, columna diagonal, o cuando se produce un empate porque todas las están ocupadas.

X = [[1, 1, 1],
[0, 0, 0],
[0, 0, 0]]

O = [[0, 0, 0],
[1, 1, 0],
[0, 0, 0]]

Operación a realizar: Suma y resta de las matrices X y O, suma de elementos de filas, columnas y diagonales para determinar si existe un jugador, y comparación de matrices para comprobar las posiciones ocupadas y detectar si el tablero esta completo.

4.2. Procedimiento paso a paso

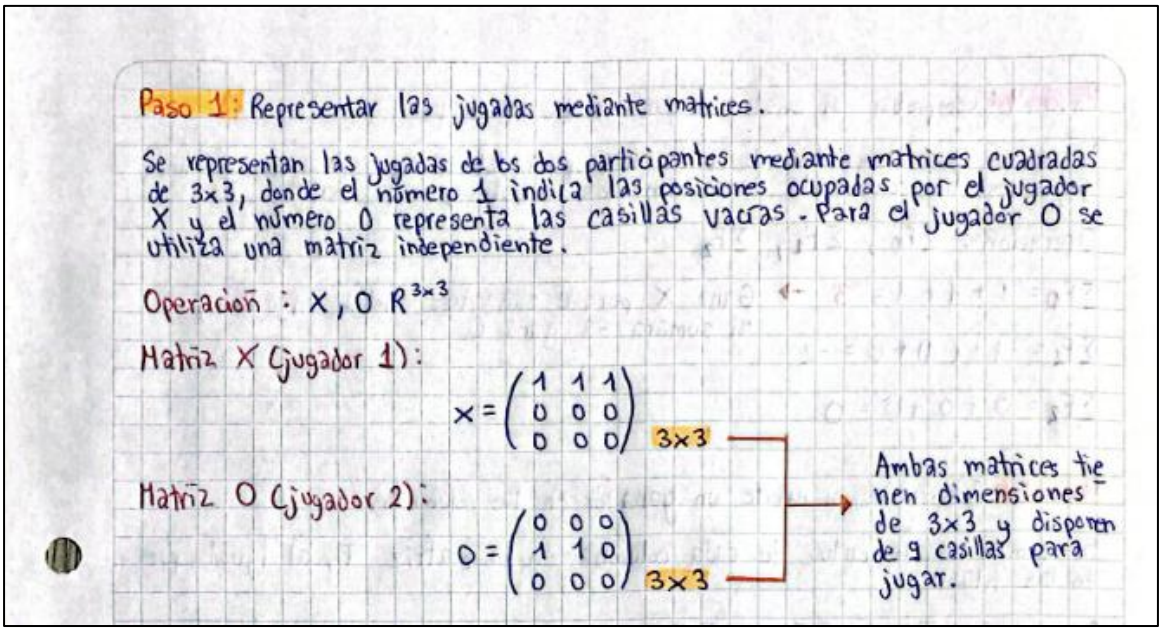


Figura 1. Representar las jugadas mediante matrices.

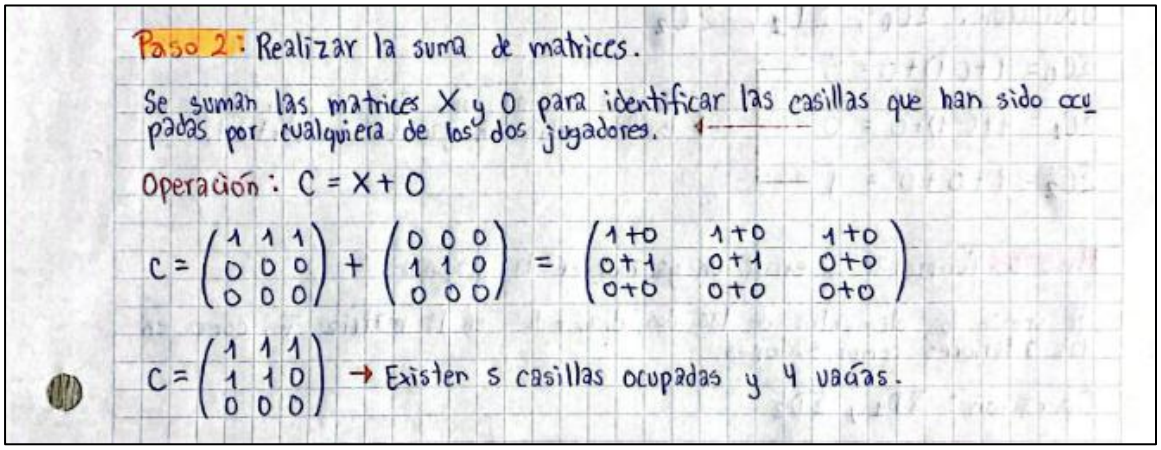


Figura 2. Realizar la suma de matrices.

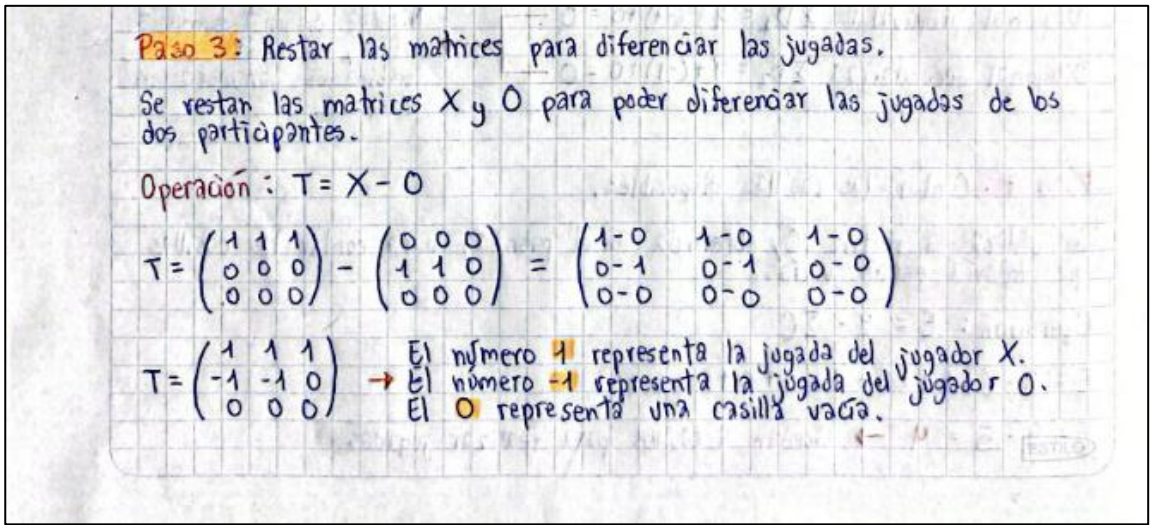


Figura 3. Restar las matrices para diferenciar las jugadas.

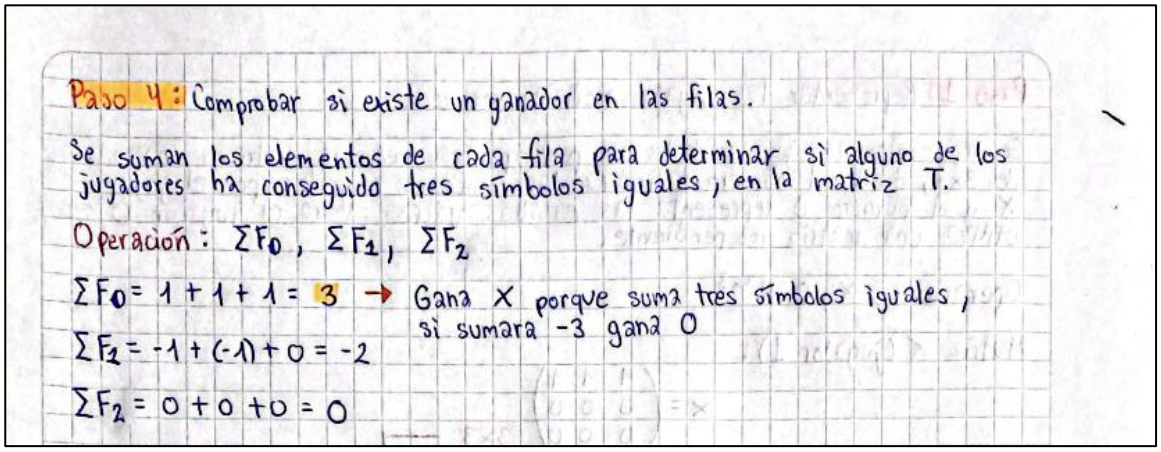


Figura 4. Comprobar si existe un ganador en las filas.

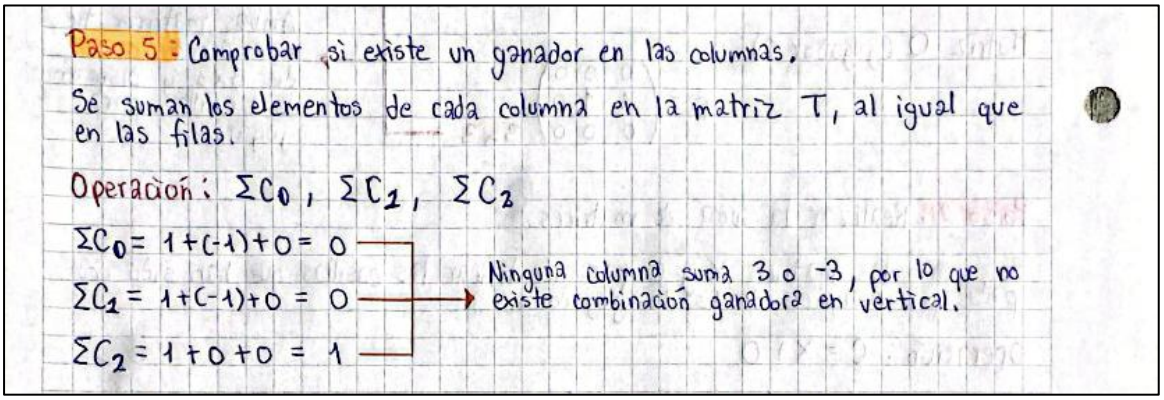


Figura 5. Comprobar si existe un ganador en las columnas.

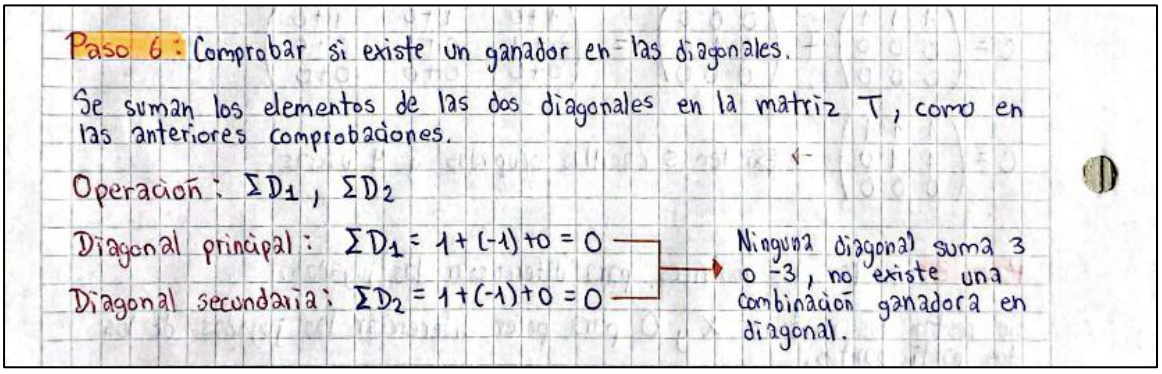


Figura 6. Comprobar si existe un ganador en las diagonales.

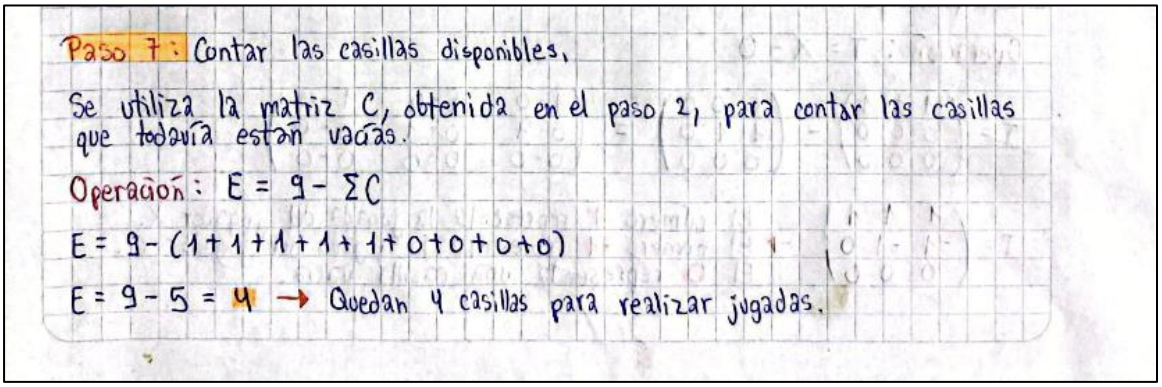


Figura 7. Contar las casillas disponibles

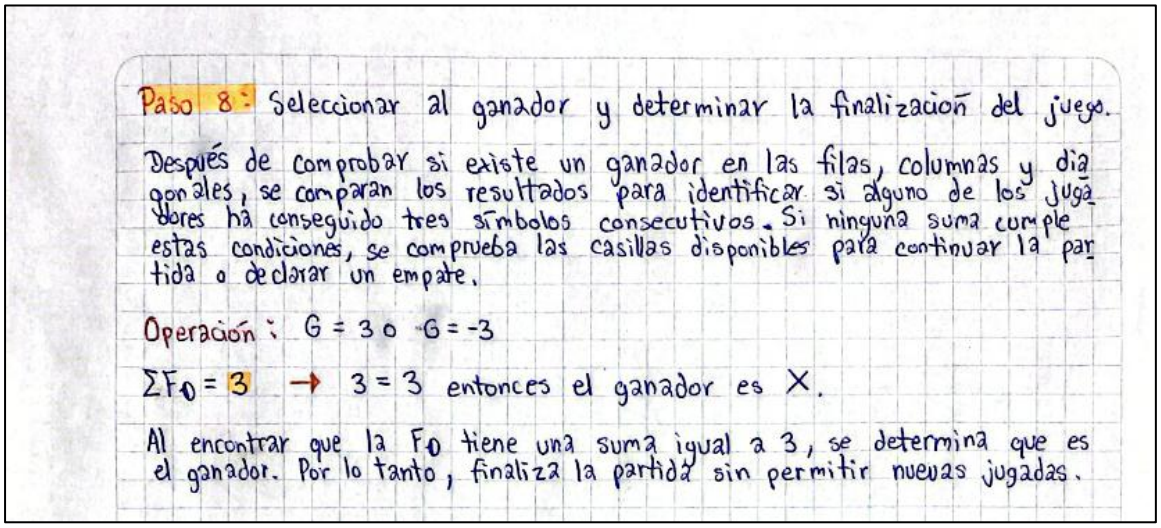


Figura 8. Seleccionar al ganador y determinar la finalización del juego.

4.3 Verificación de resultados

Para comprobar que las operaciones de matrices funcionan correctamente, se utilizan los siguientes valores correspondientes a una partida de 3 en raya.

1. **Asignación de las jugadas.**

   El jugador X realiza sus jugadas en las posiciones (1,0), (2,0), (0,2) y (2,1). El jugador O ocupa las posiciones (0,0), (2,2) y (1,1).

X = [[0, 0, 1],
[1, 0, 0],
[0, 1, 0]]

O = [[1, 0, 0],
[0, 1, 0],
[0, 0, 1]]

2. **Verificación mediante la suma de matrices**

C = X + O

C = [[0, 0, 1], + [[1, 0, 0], = [[0+1, 0+0, 1+0],
[1, 0, 0], [0, 1, 0], [1+0, 0+1, 0+0],
[0, 1, 0]] [0, 0, 1]] [0+0, 1+0, 0+1]]

C = [[1, 0, 1],
[1, 1, 0],
[0, 1, 1]]

La matriz contiene 6 unos y 3 ceros Esto significa que hay 6 castillas ocupadas y 3 disponibles

3. Verificación mediante la diferencia de matrices

T = X - O

T = [[0, 0, 1], - [[1, 0, 0], = [[0-1, 0-0, 1-0],
[1, 0, 0], [0, 1, 0], [1-0, 0-1, 0-0],
[0, 1, 0]] [0, 0, 1]] [0-0, 1-0, 0-1]]

T = [[-1, 0, 1],
[1, -1, 0],
[0, 1, -1]]

Se comprueba que los valores 1 corresponden al jugador X, los valores -1 al jugador O y los valores cero a las vacías.

4. Verificación de la condición de ganador

Filas:

$\sum F_0 = 1 + 0 + 1 = 0$

$\sum F_1 = 1 + (-1) + 0 = 0$

$\sum F_2 = 0 + 1 + (-1) = 0$

Ninguna fila suma 3 o -3, por lo tanto, no existe un ganador horizontal.

Columnas:

$\sum C_0 = -1 + 1 + 0 = 0$

$\sum C_1 = 0 + (-1) + 1 = 0$

$\sum C_2 = 1 + 0 + (-1) = 0$

Ninguna columna suma 3 o -3, por lo tanto, no existe un ganador vertical.

Diagonales:

$\sum D_1 = -1 + (-1) + (-1) = -3$

$\sum D_2 = 1 + (-1) + 0 = 0$

La diagonal principal suma -3, lo que indica que el jugador O ha conseguido tres símbolos consecutivos.

5. Verificación de las casillas disponibles

$E = 9 - \sum C$

$E = 9 – (1 + 0 + 1 + 1 + 1 + 0 + 0 + 1 + 1)$

$E = 9 – 6 = 3$

Quedan 3 casillas disponibles para realizar nuevas jugadas, en el caso de que no se complete la condición.

6. Selección del ganador

Después de comprobar las filas, columnas y diagonales, se aplica la condición para seleccionar el ganador.

G = 3 o **G = -3**

En la diagonal principal salió -3, entonces **-3 = -3.**

Esto indica que el jugador O es el ganador porque ha conseguido los tres símbolos y la suma de la D_1 es -3. Por esta razón, el juego finaliza inmediatamente, aunque todavía existan 2 casillas disponibles.

5. Procedimiento / Metodología

5.1 Descripción de la aplicación

La aplicación consiste en un juego de Tres en Raya desarrollado en Python y ejecutado en Google Colab. Está dirigido a estudiantes y personas que deseen jugar una partida entre dos participantes. Su objetivo principal es permitir que dos jugadores coloquen sus símbolos en un tablero de 3x3, validando las posiciones seleccionadas y determinando automáticamente cuándo existe un ganador o un empate.

El tablero se representa mediante una matriz de NumPy, donde el número 1 representa al jugador X, el -1 al jugador O y el 0 las casillas vacías. La aplicación utiliza operaciones de matrices, indexación y suma de elementos para identificar las jugadas, comprobar las condiciones del ganador y determinar cuándo se han ocupado todas las posiciones.

5.2 Descripción de la aplicación

Categoría	Herramienta / Tecnología	Versión / Detalle
Lenguaje de programación	Python	3.13.18
Entorno de desarrollo (IDE)	Google Colab	
Librerías / Frameworks	NumPy	2.4.6
Plataforma de ejecución	Web	

5.3 Funcionalidades implementadas

- **Funcionalidad 1: Representación del tablero.**
  Permite visualizar el tablero 3x3 casillas y registrar las posiciones de cada jugador.
  — Aplica: matrices y elementos matriciales.
- **Funcionalidad 2: Registro de jugadas.**
  Almacena las posiciones seleccionadas por los jugadores X y O, asignando 1 y -1 para diferenciarlos, mientras que 0 representa las casillas vacías.
  — Aplica: matrices y asignación de valores.
- **Funcionalidad 3: Identificación de casillas ocupadas.**
  Realiza la suma de las matrices de los jugadores para identificar las posiciones utilizadas y las disponibles.
  — Aplica: suma de matrices.
- **Funcionalidad 4: Diferenciación de jugadores.**
  Realiza la resta de las matrices para distinguir las jugadas de X y O mediante valores positivos y negativos.
  — Aplica: resta de matrices.
- **Funcionalidad 5: Comprobación de combinaciones ganadoras.**
  Suma los elementos de las filas, columnas y diagonales para identificar si algún jugador consiguió tres símbolos consecutivos.
  — Aplica: suma de elementos matriciales.
- **Funcionalidad 6: Determinación del ganador.**
  Analiza los resultados obtenidos: si una suma es 3 gana X, si es -3 gana O. Si no hay ganador, comprueba si existen casillas disponibles para continuar o declarar empate.
  — Aplica: operaciones matriciales y condiciones lógicas.
- **Funcionalidad 7: Finalización de la partida.**

Detiene el juego cuando se identifica un ganador o cuando el tablero está lleno sin existir una combinación ganadora.
— Aplica: análisis de matrices y condiciones lógicas.

5.4 Capturas de pantalla

```
==================================
 ¡Bienvenido al juego de Tres en Raya!
==================================
   |   |
---------
   |   |
---------
   |   |
---------
Turno del jugador: X
Fila (1-3): 1
Columna (1-3): 1
```

Figura 9. Presentación del tablero inicial.

```
X |   |
---------
  |   |
---------
  |   |
---------
Turno del jugador: O
Fila (1-3): 2
Columna (1-3): 2
X |   |
---------
  | O |
---------
  |   |
---------
Turno del jugador: X
Fila (1-3): 1
Columna (1-3): 2
X | X |
---------
  | O |
---------
  |   |
---------
Turno del jugador: O
Fila (1-3): 2
Columna (1-3): 3
X | X |
---------
  | O | O
---------
  |   |
---------
Turno del jugador: X
Fila (1-3): 1
Columna (1-3): 3
```

Figura 10. Registro de las jugadas realizadas por los dos jugadores.

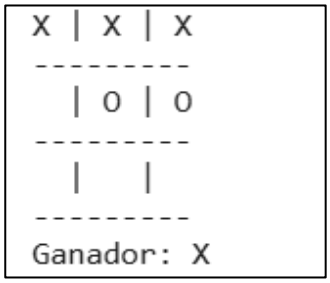


Figura 11. Mensaje de victoria cuando un jugador consigue tres símbolos consecutivos.

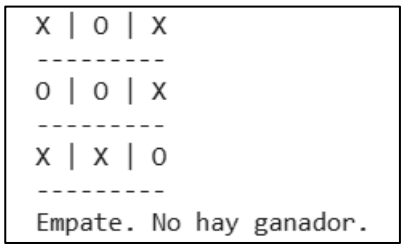


Figura 12. Mensaje de empate cuando se ocupan todas las casillas sin conseguir un ganador.

5.5 Código fuente (fragmentos clave)

```
import numpy as np

# Crear tablero de 3x3
tablero = np.zeros((3, 3), dtype=int)

# Mostrar tablero
def mostrar_tablero():
    simbolos = {0: " ", 1: "X", -1: "O"}

    for fila in tablero:
        print(" | ".join(simbolos[x] for x in fila))
        print("---------")

# Comprobar ganador
def comprobar_ganador():
    # Sumar filas y columnas
    for i in range(3):
        suma_fila = np.sum(tablero[i, :])
```

```
        if abs(suma_fila) == 3:
            return 1 if suma_fila == 3 else -1

        suma_col = np.sum(tablero[:, i])
        if abs(suma_col) == 3:
            return 1 if suma_col == 3 else -1

    # Comprobar diagonales
    d1 = np.trace(tablero)
    if abs(d1) == 3:
        return 1 if d1 == 3 else -1

    d2 = np.trace(np.fliplr(tablero))
    if abs(d2) == 3:
        return 1 if d2 == 3 else -1

    return 0

# Juego principal
print("====================================")
print(" ¡Bienvenido al juego de Tres en Raya! ")
print("====================================")

jugador = 1

while True:
    mostrar_tablero()

    print("Turno del jugador:",
          "X" if jugador == 1 else "O")

    fila = int(input("Fila (1-3): ")) - 1
    columna = int(input("Columna (1-3): ")) - 1

    if not (0 <= fila < 3 and 0 <= columna < 3):
        print("Posición fuera del tablero.")
```

```
        continue

    if tablero[fila, columna] != 0:
        print("Casilla ocupada. Intente nuevamente.")
        continue

    # Registrar jugada
    tablero[fila, columna] = jugador

    ganador = comprobar_ganador()

    if ganador != 0:
        mostrar_tablero()
        print("Ganador:",
              "X" if ganador == 1 else "O")
        break

    if np.all(tablero != 0):
        mostrar_tablero()
        print("Empate. No hay ganador.")
        break

    # Cambiar jugador
    jugador *= -1
```

Repositorio / Enlace al proyecto:

https://colab.research.google.com/drive/1D0iJeAjGa6mnKuZLcN_1TKvhFw9CJIwp#scrollTo=pLcEc9OJIUSO

6. Resultados

- ¿Los resultados del cálculo manual coinciden con los de la aplicación? ¿Qué diferencias existen y a qué se deben?

  Si, los resultados coinciden porque tanto en el procedimiento manual como en la aplicación se utilizan matrices para representar las jugadas. La suma y la resta de matrices permiten identificar las posiciones ocupadas, mientras que el programa utiliza suma de filas, columnas y diagonal para detectar al ganador. La diferencia es que manualmente desarrollan las operaciones paso a paso, mientras que Python realiza los cálculos automáticamente.

• ¿Cuáles fueron las mayores dificultades que encontraste al resolver el problema manualmente? ¿Y al programarlo?

La mayor dificultad que encontré en el procedimiento manual fue realizar correctamente las operaciones entre matrices y comprender cómo identificar al ganador mediante la suma de sus elementos. Al programarlo, la dificultad principal fue controlar las posiciones del tablero y establece las condiciones necesarias para validar las jugadas.

• ¿En qué situaciones reales de la computación o ingeniería se aplica el tema que estudiaste?

Las matrices se utilizan en el procesamiento de imágenes, los gráficos computacionales, la inteligencia artificial y el almacenamiento de información. También permiten representar tableros, organizar datos y desarrollar aplicaciones que requieren trabajar con posiciones y estructuras bidimensionales.

• ¿Qué aprendiste al construir la aplicación que no hubieras aprendido solo con teoría?

Aprendí a representar un problema mediante matrices y a utilizar operaciones de Álgebra Lineal para desarrollar una aplicación funcional. También pude comprender cómo utilizar la biblioteca NumPy para realizar cálculos, acceder a posiciones específicas y comprobar condiciones mediante programación.

7. Conclusiones

- **Primera conclusión:** La representación del juego Tres en Raya mediante una matriz de 3x3, permitió organizar las jugadas de los dos participantes de manera ordenada, identificando las castillas ocupadas y disponibles. Esto demostró la utilidad de las matrices para almacenar y representar información dentro de un programa.
- **Segunda conclusión:** La aplicación de la suma y resta de matrices facilitó la identificación de las jugadas de cada participante. Además, mediante la suma de las filas, columnas y diagonales, fue posible establecer las condiciones para detectar al ganador y determinar cuándo debía finalizar la partida.
- **Tercera conclusión:** El desarrollo de la aplicación en Python utilizando la biblioteca NumPy, permitió automatizar las operaciones matriciales y las condiciones del juego. Con este proyecto se comprobó que los conocimientos de Álgebra Lineal no sólo tienen una aplicación matemática, sino que también son fundamentales para resolver problemas y desarrollar soluciones en computacionales.

8. Referencias

[1] D. C. Lay, S. R. Lay, and J. J. McDonald, *Linear Algebra and Its Applications*, 6th ad. Hoboken, NJ, USA: Pearson, 2021.

[2] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, no.7825, pp. 357-362, Sep. 2020, doi: 10.1038/s41586-020-2649-2.

[3] NumPy Developers, “NumPy documentation,” *NumPy.org.* Disponible en: https://numpy.org/doc/stable/.